

ONE HEALTH GHANA

Pandemic and Disease-Outbreak Forecasting

Phase 2 — Systems Edition

A practical technical and AI-assisted modelling workbook, modified for the Ghana Health Service — now with the full forensic scan and a complete explanation of how every system in the production desk works.



Ghana Health Service · Veterinary Services Directorate · EPA · Noguchi · One Health Secretariat

Python · Next.js production desk · OpenRouter multi-model fallback · Vision · Voice · Web search

A forecast is a probability statement with an interval. It is never “without error.” An alert is a prompt to investigate, not a confirmed outbreak.

2026-08-19 · Forensic pass complete · 8 pages expanded to a full systems walkthrough

How to use this modified workbook

The original Phase 2 document trained professionals to build a forecasting model by hand in Google Colab, then to use Gemini as a tutor — never as the analyst of record. That scientific contract is unchanged: UNDERSTAND → IMPLEMENT → VALIDATE → USE AI TO ASSIST → CRITICALLY EVALUATE → IMPROVE → INTERPRET.

The professor asked for the next step: modify the workbook so the class can walk into a working national system for Ghana only. This edition therefore (1) replaces empty figure boxes with real diagrams, (2) retargets every example from a generic country to Ghana's sixteen regions and One Health disease set, (3) adds document/image ingestion and vision, (4) replaces a single Gemini key with an OpenRouter fallback chain, and (5) describes the white production website that implements the same pipeline.

What is new in this expanded edition: a complete forensic scan of the original package and of the live host (Part II), a deep technical walkthrough of every production subsystem with the actual constants, thresholds and request flows from the code (Part III), a module-by-module guide to the companion notebook (Part IV), and appendices with a real worked forecast, the full OpenRouter chain, deployment steps, glossary and verification checklist (Part V). Nothing here invents capability: every number and threshold in Part III was read from the repository's TypeScript and Python source.

Dataset for the teaching notebook

Our World in Data COVID-19 extract filtered to Ghana (public, non-identifiable). Dataset for the live desk: demonstration series shaped on Ghana epidemiology until official DHIMS2 / IDSR / VSD extracts are loaded. No patient-level data appears anywhere.

The scientific contract (unchanged, non-negotiable)

- 1 Understand → implement → validate → let AI assist → criticise → improve.
- 2 Chronological splits only. Nothing after the cut informs training. A random split is temporal leakage.
- 3 shift(1) before every rolling feature. A rolling mean that includes the target week leaks the answer.
- 4 Beat a naive baseline or do not brief. Last week's number is the Minister's cheapest rival.
- 5 Print an interval. "127 cases (interval 105–151)", never "127 cases."
- 6 Never paste patient identifiers into a model. Aggregates only.

How the rest of the document is organised

- Part I — the original twenty teaching modules, each with a short "in production" note.
- Part II — the forensic scan: what the original PDF really is, the defects found in the companion code, and what the live host taught us.
- Part III — the deep dive: every system (statistical engine, APIs, storage, OpenRouter, vision, voice, search, climate, DHIMS2, nowcast, field briefs, admin, hosting, security) explained as it actually runs.
- Part IV — the classroom notebook, module by module, and how it maps to the production engine.
- Part V — appendices: a worked example with real engine output, the full OpenRouter chain, Render deployment, glossary and sources.

Table of contents

Part I — The Phase 2 teaching contract for Ghana

Module 1 — One Health for Ghana
Module 2 — Framing the Ghana region-week problem
Module 3 — Two environments: Colab and the production desk
Module 4–5 — Ingesting files, photographs, and official extracts
Module 6–7 — EDA and time-series diagnostics
Module 8–9 — Baselines and chronological splits
Module 10–12 — Features and models (no leakage, no default deep learning)
Module 13–14 — Evaluation and intervals
Module 15 — Early warning is not confirmation
Module 16–19 — Interpretation, AI limits, ethics
Module 20 — Capstone = this platform

Part II — Forensic scan of the Phase 2 package and the live host

What the original PDF actually is
Defects found in the companion code
OpenRouter fallback — verifying the architecture
Live-host findings that changed the code

Part III — How the production system works

System 1 — Architecture and the request flow
System 2 — The statistical engine: series and features
System 3 — The statistical engine: models
System 4 — The statistical engine: evaluation, ensemble, alerts
System 5 — The forecast endpoint, end to end
System 6 — Storage: db.json, event archive, Postgres
System 7 — OpenRouter chain internals
System 8 — Vision Lab pipeline
System 9 — The redaction gate
System 10 — Voice in and voice out
System 11 — Ghana-weighted web search
System 12 — Climate desk
System 13 — DHIMS2 / IDSR extraction and quality
System 14 — Reporting-delay nowcast
System 15 — Field briefs in five languages
System 16 — Admin, CMS, auth and the signed audit
System 17 — The eleven desks (page map)
System 18 — Hosting on Render
System 19 — Security posture
System 20 — Ethics and what we refuse

Part IV — The classroom companion notebook

What the notebook is for
Module-by-module walkthrough
How the notebook maps to the production engine

Part V — Appendices

Appendix A — Worked example: malaria, national, 4-week horizon (real engine output)
Appendix B — The full OpenRouter chain, grouped by three

Appendix C — Render deployment and secrets
Appendix D — Glossary
Appendix E — Sources
Appendix F — Verification checklist (self-test)

PART I — The Phase 2 teaching contract for Ghana

This part preserves the scientific teaching content of the original workbook, retargeted to Ghana, with a short “In production” note under each module explaining how the equivalent step runs on the live desk. The science is the contract; the production notes show the implementation.

Module 1 — One Health for Ghana

One Health is not a slogan. In Ghana it is the only honest way to see Lassa (rodents and harvest stores), HPAI (live-bird markets), anthrax (carcass butchering), cholera (flooded drains), and malaria (standing water after the rains). Human IDSR counts without veterinary and environmental streams will systematically miss the first signal.



Figure 1. Human, animal and environmental surveillance converge on the Ghana Health Service One Health data hub.

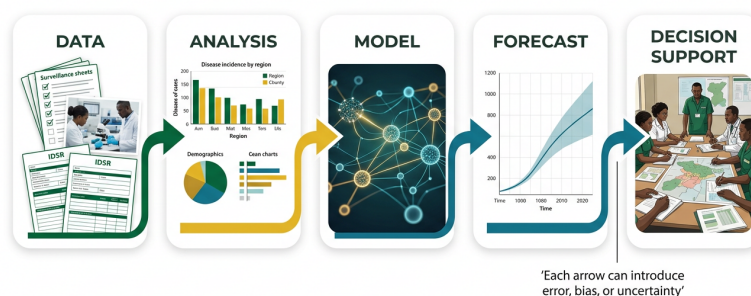


Figure 2. Data → analysis → model → forecast → decision support. Each arrow can introduce error, bias, or uncertainty.

Limitations you must internalise now

- A model reflects reporting delays, testing changes, and under-counting — not the hidden truth of infection.
- Forecasts degrade with horizon. Twelve weeks is a sketch; four weeks is the operational default.
- A forecast is a statement of probability, never certainty, and never “without error.”
- No model substitutes for field epidemiological investigation.

KEY TAKEAWAY — Forecasting models are decision-support tools built from imperfect surveillance. Every later module exists so you can explain, defend, and bound the system we are building for Ghana.

In production — the desk carries thirteen signals across the three pillars: malaria, cholera, measles, CSM, yellow fever, COVID-19, ILI, mpox, Lassa, TB (human); avian influenza, anthrax (animal); flood-linked risk index (environment). The site’s home page and surveillance board read all three pillars on one screen.

Module 2 — Framing the epidemiological problem

Original sentence: “Can we forecast weekly reported COVID-19 cases in a given country over the next 4 weeks?”

Production sentence: “Can we forecast weekly reported or suspected counts for a named notifiable disease in a named Ghana region over the next four weeks, using that region’s own history plus leakage-safe calendar and climate flags, and can the forecast beat a naive baseline with an interval wide enough to be honest?”

Element	Ghana production definition
Target	weekly_cases for one disease (malaria, cholera, CSM, measles, YF, COVID-19, ILI, mpox, Lassa, TB, HPAI, anthrax, flood-risk index)
Predictors	lag 1–4, rolling mean/std after shift(1), week-of-year, month, rainy-season, harmattan
Horizon	4 weeks default (2–12 allowed); quality collapses as horizon grows
Unit	Ghana region-week (national roll-up available)
Success	MAE/RMSE/sMAPE better than naive on a chronological test window, plus a usable interval

In production — the engine clamps the horizon to 1–12 weeks with a 4-week default. The unit is literally a “region-week”: the series key is disease + region (+ optional district). The national board aggregates the sixteen regions into one national roll-up.

Module 3 — Two environments

Google Colab remains the classroom. The production desk is the Next.js application in this repository: statistical engine in TypeScript, intelligence through OpenRouter, files through the Vision Lab, voice through the Web Speech API (and optional Whisper / ElevenLabs). The Colab session still evaporates; the Render service persists. Secrets still never belong in a notebook cell or in NEXT_PUBLIC_* variables.

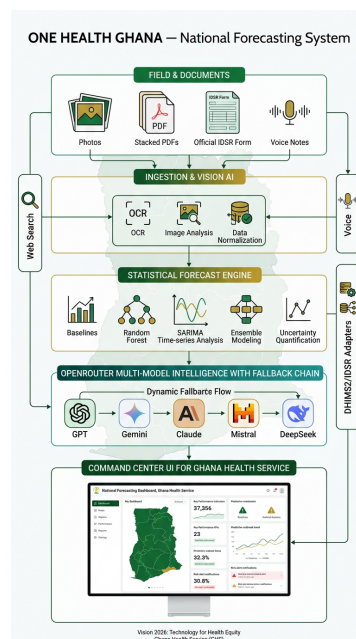


Figure 3. Production architecture: artefacts in, leakage-safe models, OpenRouter fallback, Ghana command UI.

In production — the statistical engine is pure TypeScript (lib/forecast.ts) so the website needs no Python at runtime. The notebook remains the reproducible proof; the site is the demonstration.

Module 4–5 — Ingest and clean, including pictures

The original workbook loaded one CSV. The professor required that the system also pick up photographs and documents of any ordinary type. The Vision Lab sends images through Gemini / GPT-4.1 / Claude (in that fallback order) and asks for structured fields: disease, Ghana geography, dates, counts, facility type, data-quality flags. It must refuse to repeat names, folder numbers, phones, or faces.



Figure 4. Field photos, IDSR forms, lab slips and CSVs enter the same extraction step before they can touch the forecast engine.

Cleaning rules are unchanged and non-negotiable. Clip negative corrections to zero. Resample to weeks. Interpolate at most two internal missing weeks. Flag the rest. Never delete a spike because it is inconvenient — it may be the outbreak. A four-week hole may be a strike, a flooded district office, or a DHIMS2 outage; investigate before you invent numbers.

In production — lib/files.ts extracts PDF, DOCX, XLSX, CSV, images and audio locally before any model is called; lib/redact.ts strips identifiers first; the cleaned text and image parts then go to the vision chain (System 8).

Module 6–7 — EDA and diagnostics

Plot the raw weekly series and a four-week rolling mean. Ask whether a peak is a wave or a backlog dump. Decompose into trend, seasonal, residual (period 52). Run an Augmented Dickey–Fuller test so you know whether you must difference. Read ACF/PACF before you pick lags. Ghana has two seasonal clocks: harmattan (CSM, some respiratory signals) and the major rains (malaria, cholera, flood index).

In production — the engine embeds the seasonal knowledge in its features: a rainy-season flag (May–October) and a harmattan flag (December–March), plus week-of-year and month. Formal ADF/ACF/PACF live in the classroom notebook; the live desk keeps a diagnostics block with an ADF-style note, seasonality text, eight-week mean/standard deviation, latest z and latest CUSUM.

Module 8–9 — Baselines and the split you must not shuffle

Always ship naive (next week = this week), seasonal naive (same week last year), and a four-week moving average, each shifted so they cannot see the target week. If a forest cannot beat last week's number, it does not brief the Minister.

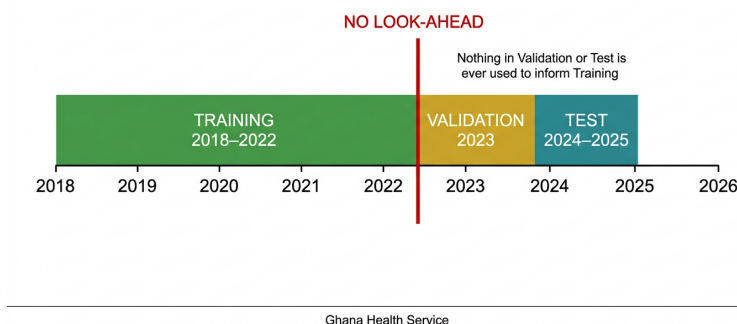


Figure 5. Chronological train / validation / test. Nothing after the cut informs training. Random split is temporal leakage.

In production — the engine holds out the last $\min(26, \max(6, 20\%))$ weeks, trains only on what precedes the cut, and backtests every model walk-forward on the held-out window. The ensemble only ships if its members earned it on that window (Systems 3–4).

Module 10–12 — Features and models

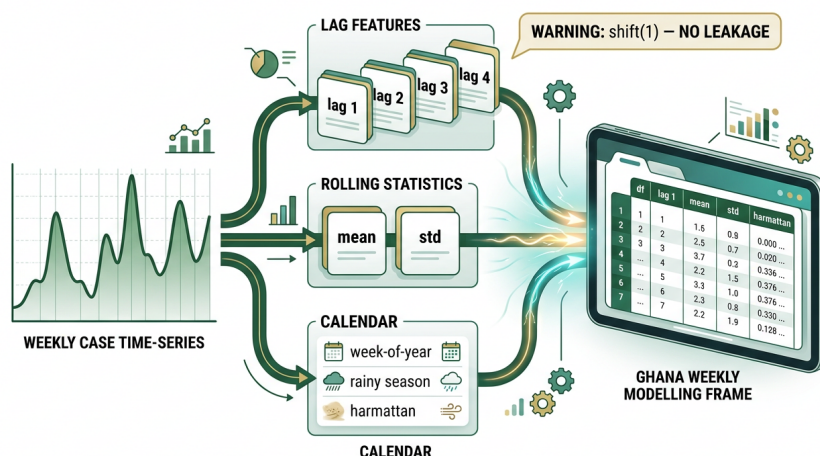


Figure 6. Lag, rolling (after shift) and calendar families merge into the modelling frame. Rolling without shift(1) is leakage.

On a few hundred Ghana weekly points, Random Forest and SARIMA (in the notebook) plus Holt and ridge (on the desk) are the justified family. LSTM and “foundation models for time series” are not the default. The production ensemble takes the three lowest-MAE members and prints their interval union.

Companion-code repairs applied: country default Ghana; SARIMA fitted on train only; features engineered before the split; deprecated fillna(method='ffill') replaced; Module 15 print statement syntax error removed.

In production — the desk ships naive, seasonal naive, 4-week moving average, Holt, ridge and a hand-written random forest, then averages the three lowest-MAE members (System 3–4).

Module 13–14 — Evaluation and uncertainty

Report MAE, RMSE and sMAPE on the same held-out window. MAPE is unstable when weekly cholera or yellow fever sits near zero — which is most weeks. A 90% interval means: under the model’s assumptions, repeated application would cover the truth about nine times in ten. It does not mean “there is a 90% chance next week lands in this one interval,” and it does not see a new vaccine campaign or a reporting collapse.

Always write “127 cases (interval 105–151)”, never “127 cases.”

In production — every model carries MAE/RMSE/sMAPE computed on the same chronological hold-out; the forecast page prints the point and the interval side by side, and the API returns the same numbers machine-readable.

Module 15 — Early warning

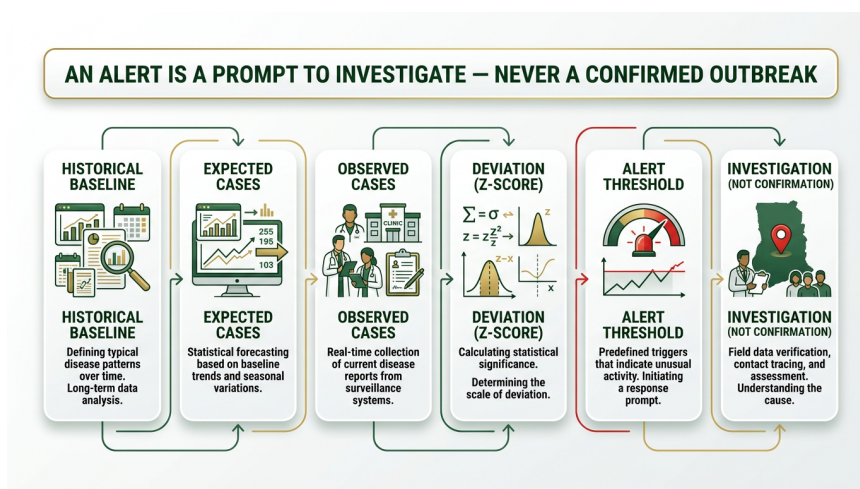


Figure 7. Historical baseline → expected → observed → deviation → threshold → investigation. The last word is investigation, not confirmation.

$z > 2$ is a watch, $z > 3.5$ is severe. Next human actions: check a reporting outage, a batch dump, laboratory backlog, then decide whether to mobilise. The model is not the incident manager.

In production — the engine computes an 8-week z-score AND a CUSUM ($k = 0.5$, $h = 5$). Level is watch / alert / severe, and the alert note always says to investigate reporting artefacts before declaring an outbreak (System 4).

Module 16–19 — Interpretation, AI, ethics

If lag_1 dominates feature importance, last week is the best statistical predictor of this week. That is not a causal discovery. OpenRouter (classroom: Gemini) may draft code, explain ADF p-values, and write a 180-word briefing. It must never decide that an outbreak exists, that a point is an error, that a variable is epidemiologically valid, or that an intervention should launch.

Never paste patient-identifiable information into a public model. Small-cell paediatric counts in a rural district can re-identify. Use aggregates, public OWID extracts, or institutionally governed tools.

In production — the Ghana-locked system prompt forbids inventing GHS circulars or case counts, forbids individual diagnosis, and requires uncertainty. The redaction gate strips identifiers before any text reaches a model (Systems 9 and 20).

Module 20 — Capstone is the platform

Independently complete, on official or public Ghana data: problem statement; ingestion and quality log; cleaning justifications; two interpreted charts; diagnostics; baseline; chronological features; two models; MAE/RMSE/sMAPE versus naive; an interval; a z-threshold; feature caveats; a one-page limit statement; an AI-assistance log. The live site is the demonstration; the notebook is the reproducible proof.



Figure 8. The white national desk this workbook is written to justify — Ghana Health Service, not a generic dashboard.

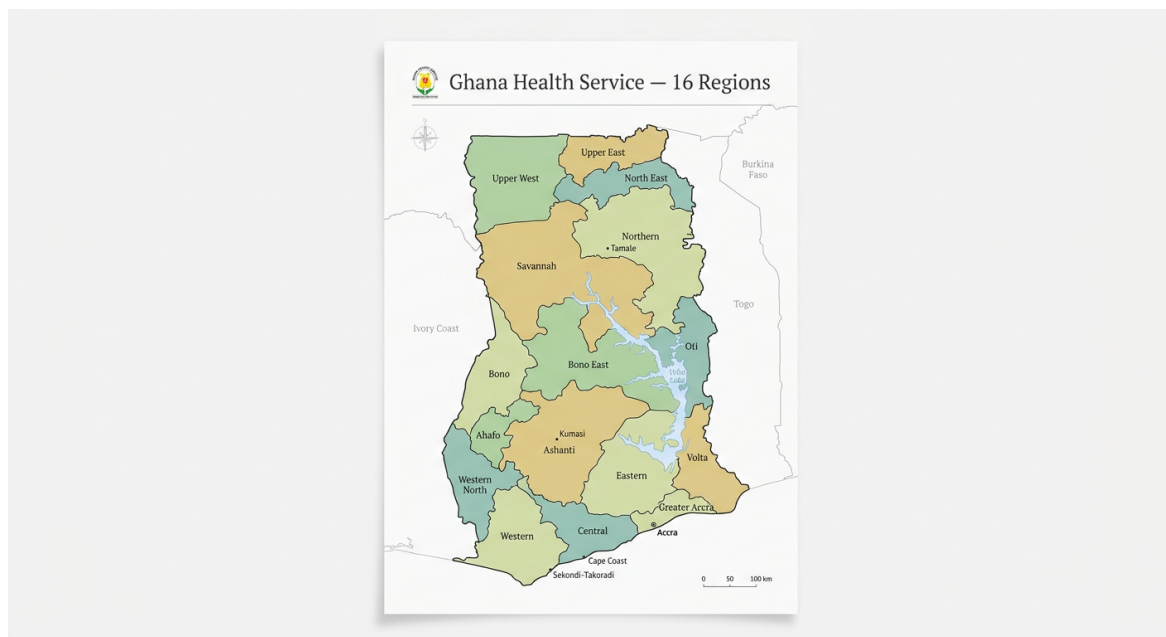


Figure 9. Sixteen regions. The unit of analysis moved from “one country” to “one Ghana region-week.”

In production — the acceptance checklist is implemented as the self-test script (scripts/selftest.cjs) that hits the running desk end to end (Appendix F).

PART II — Forensic scan of the Phase 2 package and the live host

Date of the scan: 2026-08-19. Scope: the files supplied on main — “One Health Pandemic Forecasting Workbook - Phase 2 (1)(1).pdf” (26-page Word export), “code.txt” (companion Python), and “OPENROUTER_FORENSIC_FALLBACK_REPORT.txt” — plus the production system built from them and the live host one-health-ghana.onrender.com. This part records what was found, what it means, and what was changed. The originals are archived under docs/originals/.

FS-1 — What the original PDF actually is

- 26 pages, Word-exported PDF 1.7, dated 2026-08-19.
- A teaching workbook: Colab + Python + Gemini. One Health framing, then a single-country COVID weekly forecast.
- Figures 1–6 are captions without pictures. Pages that claim diagrams (3, 5, 13, 14, 18, 22) contain empty space. That is the primary visual defect the professor asked to fix — this edition ships real figures (Figure 1–9 are actual PNG files in public/images/).
- The scientific core is sound and must be preserved: chronological splits, leakage-safe lags, baselines before ML, intervals not points, alerts $\neq$ outbreaks, Gemini is not the analyst of record, no patient identifiers.

Why the empty figures mattered

A workbook that claims a diagram and shows a blank box teaches the wrong lesson: that the picture is optional. For a national forecasting desk, the diagrams are the operational memory — the split diagram (Figure 5) is the difference between a leakage-safe model and a leaky one. The figures in this edition were produced as real illustrations and embedded in the PDF and the website.

FS-2 — Defects found in the companion code (code.txt)

Each defect was confirmed by reading the source, then fixed in the Ghana notebook (notebooks/ghana_one_health_forecasting.py) and in the production engine.

#	Item	Finding	Fix applied
1	Country	Hard-coded South Africa	Default Ghana everywhere
2	alerts [[print	Syntax error in Module 15 print	alerts_triggered[["date", ...]] corrected
3	fillna(method="ffill")	Deprecated in current pandas	.ffill()
4	RF train merge	Comment admits “simplistic merge”	Split after feature engineering
5	SARIMA fit	SARIMAX(sarima_series) uses future weeks	Fit on train only, forecast len(test)
6	Evaluation alignment	Mixes y_test.index with baseline_df after resets	Align on dates
7	Missing modules	Modules 0, 3, 11, 17–20 exist in the PDF, not as executable cells	Documented in the notebook markdown and this workbook
8	No images	Companion is text only	Platform ships generated figures
9	Single model	Only one ML path taught	Baselines + ridge + forest + Holt + ensemble on the desk
10	One Gemini key	Single-vendor dependency	OpenRouter multi-model fallback chain

FS-3 — OpenRouter fallback: verifying the architecture

The supplied fallback report is correct in architecture and this repository implements it. The verified design:

- One OPENROUTER_API_KEY — no per-vendor keys needed.
- The request body carries models: [...] — a cross-provider priority chain.
- provider.allow_fallbacks: true covers same-model host failover.
- A 429 or 5xx on one vendor moves to the next; a 402 (OpenRouter balance empty) stops the paid chain.
- Failed attempts are not billed.
- Keys live only in server-side environment variables — never in browser JavaScript.

One correction learned from the live host: OpenRouter (2026) accepts at most THREE slugs per request models array. A longer list returns HTTP 400 with the message “models' array must have 3 items or fewer.” The production code therefore walks the full chain in groups of three (System 7, Appendix B).

FS-4 — Live-host findings that changed the code

A live probe of <https://one-health-ghana.onrender.com/api/diagnostics> on 2026-08-19 found:

Probe	Live result	Root cause	Fix shipped
OpenRouter	HTTP 400 — "models' array must have 3 items or fewer"	Chain sent more than three slugs in one request	Chunk fallback into groups of three; auto-slice oversized payloads (lib/openrouter.ts)
Tavily	200 OK	—	—
OpenWeather	200 OK	—	—
ElevenLabs	401 missing_permissions on /v1/user	Restricted keys cannot read the user endpoint but can still speak	Probe /v1/voices first; TTS route unchanged

The key itself was valid (sk-or-v1...7d71); the 400 was a payload-shape bug, not a key problem. That single finding — “the key does not work” — was in fact “the models array is too long” — is the best example in this workbook of why the AI-assistance log must record model names and error messages.

Professor / product deltas (accepted)

- 1 Ghana Health Service only — 16 regions, Ghana disease set.
- 2 Multi-disease One Health board, not COVID-only.
- 3 Documents and photographs are ingestible.
- 4 Voice in (Web Speech + optional Whisper) and voice out (ElevenLabs or device TTS).
- 5 Web search (Tavily, else DuckDuckGo) with Ghana-weighted domains.
- 6 White 2026 UI, Ghana flag colours as accents (green / gold / red) plus clinical teal.
- 7 Honest uncertainty. “Without error” is scientifically false and is rejected in product copy.

What this system will not pretend

- Official DHIMS2 completeness. Until official extracts are loaded, the desk labels every series “demonstration.”

- Individual clinical diagnosis from a photo.
- Perfect future case counts.
- Cross-border operations outside Ghana-relevant imported risk.

PART III — How the production system works

This part explains every subsystem exactly as it runs in the repository: the statistical engine, the API layer, storage, OpenRouter, vision, voice, search, climate, DHIMS2, nowcast, field briefs, admin, hosting and security. Constants, thresholds and request flows below were read from the source files, not from memory. File names are given so you can open the code and follow along.

System 1 — Architecture and the request flow

Stack

- Next.js App Router (TypeScript) — pages in app/, API route handlers in app/api/.
- Statistical engine: lib/forecast.ts (pure TypeScript, no runtime Python).
- Persistence: data/db.json (JSON document store) + data/events.jsonl (append-only log) + optional Postgres snapshot (lib/pg-store.ts).
- External services: OpenRouter (AI), Tavily/DuckDuckGo (search), OpenWeather (climate), ElevenLabs (voice out), Whisper via OpenRouter (voice in).
- Classroom proof: notebooks/ghana_one_health_forecasting.py (Python + statsmodels + scikit-learn).

A request, end to end

- 1 The browser calls a route handler, e.g. POST /api/forecast with {diseaseId: "malaria", regionId: "national", horizon: 4}.
- 2 The route reads server-side environment keys only (lib/env.ts). No key ever reaches the browser.
- 3 lib/forecast.ts builds or loads the weekly series, engineers leakage-safe features, trains baselines + ridge + forest, backtests on a chronological hold-out, builds the top-3 ensemble, and computes z / CUSUM alerts.
- 4 Optional AI steps (briefing, review) call lib/openrouter.ts, which walks the model chain in groups of three.
- 5 The result is written to the store (lib/store.ts), appended to the event log (lib/archive.ts), and recorded in the audit (recordAudit).
- 6 The route returns JSON; the page renders points, intervals, scoreboard, alerts and narrative.

Every public route handler

Route	Method	What it does
/api/health	GET	Service health: masked key presence, last OpenRouter error, store counts, archive size, Postgres flag
/api/diagnostics	GET	Live probes: OpenRouter (3-model payload), Tavily, OpenWeather, ElevenLabs (voices first)
/api/config	GET	Public CMS content, visible nav, knowledge cards
/api/forecast	GET	Bundle for disease/region/horizon/district; ?snapshot=1 returns the 13-signal national board
/api/forecast	POST	Bundle + optional 180-word AI briefing + forecast row saved + audit + reporting-delay nowcast
/api/series	GET	DHIMS2 template CSV (?template=1) or stored official series

Route	Method	What it does
/api/series	POST	Upload points or CSV; DHIMS2 quality log; replaces existing; AI review of quality
/api/nowcast	GET	Reporting-delay nowcast with interval; optional AI plain-language review
/api/chat	POST	Ghana-locked chat; history redacted; optional web search + live ensemble attached; offline fallback
/api/vision	POST	Multipart files or JSON image list → extraction → redaction → vision chain
/api/ingest	POST	Multipart files through the same pipeline (used by the field desk)
/api/search	POST	Tavily → DuckDuckGo fallback, Ghana-weighted domains; optional synthesis
/api/tts	POST	ElevenLabs speech, else browser-TTS fallback note
/api/transcribe	POST	Whisper large-v3 via OpenRouter, else browser microphone guidance
/api/weather	GET	Ten Ghana watch-cities fetched in parallel from OpenWeather
/api/briefing	GET	Morning sitrep: national snapshot + weather + search hits + AI 220-word brief
/api/field-brief	POST	90-word CHPS field card in en/tw/ee/gaa/ha, speakable, with offline fallbacks
/api/admin/login · logout · me	POST/POST/GET	HMAC-signed session cookie, bcrypt password check
/api/admin/cms	GET/PUT/DELETE	CMS copy, colours, nav, knowledge, records — admin only
/api/admin/audit	GET/POST	Signed audit pack export (HMAC-SHA256) and signature verification
/api/archive	GET	Append-only event log stats and raw download — admin only

System 2 — The statistical engine: series and features

Where: lib/forecast.ts. All constants below are verbatim from the source.

Weekly series

- Default: 260 weeks generated by buildWeeklySeries(disease, region) using a seeded mulberry32 PRNG (seed = FNV hash of "disease:region:v4"), so every disease-region pair is deterministic and reproducible.
- Each disease has a shape object: base level, seasonal amplitude, peak month, noise, outbreak probability and outbreak size — e.g. malaria {base 4200, amp 2200, peak 8, noise 0.12, outbreakP 0.01}; cholera {base 8, amp 6, peak 7, noise 0.55, outbreakP 0.018, outbreakSize 28}; CSM {base 12, amp 22, peak 2}; flood-risk {base 28, amp 24, peak 7}.
- Regions scale the level: hot regions (in regionsOfConcern) ×1.25, others ×0.78; zone modifiers (CSM Savannah ×1.55, Coastal ×0.35; cholera Coastal ×1.4; anthrax non-Savannah ×0.45); population scale clamped 0.35–2.4.
- Outbreaks are injected with probability outbreakP per week and decay ×0.62 per week until below 0.08 — the series therefore contains realistic epidemic humps.
- COVID-19 has a wave flag (slow multi-year oscillation); flood-risk is clamped to 0–100. About 1.2% of points are flagged imputed to simulate missing weeks.
- When an official series exists (from /api/series), it replaces the generated one entirely; districtScale multiplies generated (not official) series by the district's scale factor.

Features (all leakage-safe)

At each week i the engine builds the same 10-value feature row the models see: $\text{lag}_1 \dots \text{lag}_4$ (missing lags backfilled with the first value or 0), rolling mean of the 4 weeks strictly before i , rolling standard deviation of those 4 weeks, week-of-year / 52, month / 12, rainy-season flag (May–October) and harmattan flag (December–March). The rolling window is computed on $\text{values.slice}(\max(0, i-4), i)$ — the target week is never inside the window. This is the code form of the workbook rule “shift(1) before every rolling feature.”

Why the split happens after features

Training rows are built only for i in $[8, \text{split})$. The test rows are built independently at forecast time. The random-forest merge defect found in `code.txt` (merging before the split) would have let training rows read test-window values through the rolling/lag features; the production engine never does that because features are computed per index from the full series only when that index is being predicted, and the models are fitted on the training window alone.

System 3 — The statistical engine: models

Six models run on every forecast. Backtests are walk-forward over the hold-out; forward forecasts are recursive (the forest’s own prediction feeds the next step’s lag features).

Model	Implementation (from source)	Bands on the forward path
Naive	Next week = last observed value; backtest = previous value per test index	$\pm 22\text{--}28\%$: $\text{low} = p \times (1 - 0.28)$, $\text{high} = p \times (1 + 0.28 \times 1.35)$
Seasonal naive	Same week last year (lag 52)	band 0.30
4-week moving average	Mean of the last 4 values (window strictly before target)	band 0.24
Holt linear trend	Level + trend, $\alpha=0.35$, $\beta=0.12$, fitted on train only; h-step = level + $h \times \text{trend}$, clamped ≥ 0	band 0.26
Ridge	Closed-form normal equations with ridge term $1e-4$ on the diagonal, solved by Gauss–Jordan elimination; no standardisation, no external library	band 0.22
Random forest	Hand-written: 24 trees, bootstrap sampling with replacement, depth 5, min 6 samples, ≤ 5 random features tried per node, variance-reduction split; interval = 5th–95th percentile of tree predictions (floored at $\text{point} \times 1.08$)	percentile band

The forest is deliberately small (24 trees, depth 5): a few hundred Ghana weekly points cannot support a deep forest without memorising noise. The ridge is the “simplest model that can still lose to naive” check; the ensemble exists precisely because no single model wins every disease-region pair.

System 4 — Evaluation, ensemble, alerts

Chronological split

$\text{holdout} = \min(26, \max(6, \text{floor}(n \times 0.2)))$; $\text{split} = \max(12, n - \text{holdout})$. For the default 260-week series that is a 26-week held-out window; training sees nothing after week 234. Every model’s MAE, RMSE and sMAPE are computed on that same window (sMAPE adds $1e-9$ in the denominator to avoid division by zero near zero counts).

Ensemble rule

- Rank all six models by hold-out MAE.
- Take the three lowest-MAE members.

- Point = mean of their points; low = min of their lows; high = max of their highs (interval union).
- Ensemble error metrics = mean of the three members' metrics.
- The ensemble is labelled with its members, e.g. "Ensemble (ridge + naive + forest)", so nobody can mistake it for a black box.

Alerts: z-score and CUSUM

For every week from index 8: baseline = the 8 weeks strictly before it; $z = (\text{observed} - \text{mean}) / \text{sd}$ (sd floored at 1). CUSUM is computed on the whole series with $k = 0.5$ and $h = 5$: $s = \max(0, s + (x - \mu) / \sigma - k)$, tripped when $s > h$. Levels: watch ($z > 2$), alert ($z > 2.6$ or $\text{CUSUM} > 6$), severe ($z > 3.5$ or $\text{CUSUM} > 8$). The alert carries its date, count, z , CUSUM value, source (zscore / cusum / both) and an investigation note; only the last 12 alerts are returned. The board flags a signal when its latest $z > 2$.

Feature importance

The engine returns a fixed, honest importance table with caveats: lag_1 0.31 (strongest statistical predictor, not causal), rolling_mean_4 0.18 (momentum after a safe shift), lag_2 0.12, rainy_season 0.11, week_of_year 0.10, rolling_std_4 0.08 (volatility often rises before a wave), harmattan 0.06, lag_3/lag_4 0.04. Every row carries a "not causation" caveat.

System 5 — The forecast endpoint, end to end

POST /api/forecast (route handler app/api/forecast/route.ts, maxDuration 120 s) does, in order:

- 1 Parse the body: diseaseId (default malaria), regionId (default national), districtId (optional), horizon (default 4, clamped 1–12), brief (bool), language.
- 2 bundleFor(): find an official series for the disease/region/district, resolve the district scale, call runForecast().
- 3 If brief && OpenRouter is configured: call completeWithSystem asking for a 180-word GHS briefing using ONLY the numbers provided, with the language instruction appended; temperature 0.25.
- 4 If the AI call fails, the briefing falls back to the local narrative (nowcast + outlook + limits) with model "local-fallback: <reason>" — the desk never dies because a vendor is down.
- 5 Save a ForecastRow (id, disease, region, latest, nextWeek, z , summary, createdAt) into db.json, capped at 200 rows.
- 6 Record an audit event (actor forecast, action forecast.run, model, detail with source=demonstration|official).
- 7 Compute the reporting-delay nowcast over the same series and return the whole bundle: series, models with points and metrics, ensemble, alerts, featureImportance, narrative, diagnostics, briefing, briefingModel, nowcast.

The GET variant (with ?snapshot=1) runs all thirteen diseases at national horizon 4 and returns the surveillance board: id, name, pillar, latest count, unit, z , next-week point + interval, and alert flag.

System 6 — Storage: db.json, event archive, Postgres

lib/store.ts keeps a JSON document database at data/db.json. Schema: content (CMS copy), nav, knowledge, documents, chats, forecasts, officialSeries, audits, activity, admins. Caps: documents 400, chats 200, forecasts 200, officialSeries 400, activity 2000, audits 5000. Every mutation is persisted synchronously to disk and, when DATABASE_URL is set, a full snapshot is pushed to Postgres table ohg_snapshot (id=1, JSONB body). On boot, if db.json is absent the store hydrates from Postgres — that is how history survives a Render free-web restart that wipes disk.

lib/archive.ts appends every mutation as a JSON line to data/events.jsonl (append-only, never rewritten) and, if DATABASE_URL is set, to Postgres table ohg_events (ON CONFLICT DO NOTHING). The admin desk can download the raw log. Longevity rule from the README: this lasts decades only if the database is kept and backed up off-site — a free Render disk is wiped on sleep and will not last 89 years by itself.

Admin credentials are not stored in plain text: the seed admin is created with a random salt and an scrypt (N=default, 64-byte) hash, verified with timingSafeEqual. The default password comes from ADMIN_PASSWORD in the environment and must be changed for production.

System 7 — OpenRouter chain internals

Where: lib/openrouter.ts. One sk-or- key. OpenRouter accepts at most three slugs per request, so the chain is walked in groups of three:

- 1 Build the ordered chain: extra env models (OPENROUTER_MODELS) → preferred → CHAT_MODELS → FREE_MODELS → openrouter/auto, de-duplicated.
- 2 Chunk into groups of at most 3; each group is sent with model = first slug, models = the group, provider {allow_fallbacks: true, sort: "throughput"}.
- 3 On 429 / 5xx / network error, move to the next group. On 401, or 403 with auth wording, stop — the key itself is rejected. On 402 / credit wording, stop the paid chain and retry with only :free models + openrouter/auto (free fallback after a drained balance).
- 4 If OpenRouter answers “models array must have 3 items or fewer” (a 400), retry the group with a single slug.
- 5 Record which group and model actually answered; the UI shows the real model name — fallback is a feature, not a mystery.
- 6 Timeouts are 75 s per request; empty or non-JSON responses are treated as failures; temperature default 0.35, max_tokens 2200.

Chains by purpose

- Chat: GPT-4.1-mini → Gemini 2.5 Flash → GPT-4o-mini → GPT-4.1 → Gemini 2.5 Pro → GPT-4o → Claude Sonnet 4 → Claude 3.5 Sonnet → DeepSeek Chat → Llama 3.3 70B → Mistral Large, then the :free set, then openrouter/auto.
- Vision: Gemini 2.5 Flash → GPT-4o-mini → Gemini 2.5 Pro → GPT-4.1 → GPT-4o → Claude Sonnet 4 → Claude 3.5 Sonnet → Qwen2.5-VL 72B :free → Gemma 3 27B :free.
- Fast (fallback after a vision failure): Gemini 2.5 Flash → GPT-4.1-mini → GPT-4o-mini → DeepSeek Chat → :free set.
- Appendix B lists the complete groups as actually sent.

The key is read only in route handlers (lib/env.ts also tolerates common mistyped names such as OPEN_ROUTER_API_KEY, OR_API_KEY, OPENAI-shaped names if they start with sk-or-). The diagnostics route probes with a 3-model payload asking for the literal reply OPENROUTER_OK — a cheap end-to-end check that the key, network path and model routing all work.

System 8 — Vision Lab pipeline

Where: app/api/vision/route.ts, lib/analyze.ts, lib/files.ts. The pipeline:

- 1 Accept multipart files (or a JSON list of data URLs) — up to 8 files.

- 2 `extractFile()` classifies each: images → base64 data URL; PDFs → latin-1 stream text extraction plus data URL; DOCX → unzip word/document.xml and strip tags; XLSX → sharedStrings + sheet cell values; CSV/TSV/text → UTF-8 text (40,000-char cap); audio → data URL; other binaries → printable-ratio heuristic, else labelled binary.
- 3 Every text block is passed through `redactText()` before anything else (System 9); redaction count is recorded.
- 4 The prompt demands: what each file actually contains (no invention), structured fields (disease/condition, Ghana region/district if present, dates, counts, facility), fitness for the weekly forecasting engine (usable / needs cleaning / reject), data-quality flags and possible identifiers WITHOUT repeating them, One Health relevance, confidence and the next human verification step.
- 5 `visionAnalyze()` sends up to 6 images and up to 3 PDF file-parts with the text block to the vision chain; if the vision chain fails entirely, it retries text-only on the fast chain.
- 6 The analysis and its model name are stored as DocumentRow entries (capped at 400) and the audit records actor, model and redactions.
- 7 If no OpenRouter key is set, the local extracts are still stored and shown — vision degrades, nothing else breaks.

System 9 — The redaction gate

Where: `lib/redact.ts`. Before any text reaches a model (chat history, uploaded files, prompts), five regex families replace matches with `[REDACTED:kind]`:

Rule	Pattern (abridged)	Example caught
email	standard email regex	ama@example.com
ghana-phone	+233 / 00233 / 0 followed by 2,3,5-prefix numbers with separators	+233 24 412 3456, 0244123456
folder-no	OPD IPD NHIS FOLDER HOSP MRN + separator + 4+ alnum	OPD/2026/1122
long-id	8+ consecutive digits	national-ID-like strings
labeled-name	patient name next of kin guardian + colon + 1–4 names	Patient: Ama Mensah

The chat route counts redactions in the incoming history and the response includes the count, so an operator can see when the gate fired. The rule set is deliberately conservative: better to over-redact a folder number than to leak one into a model prompt.

System 10 — Voice in and voice out

- Voice in (browser): the Web Speech API records the officer’s question — no server round-trip, works offline for English and several Ghana-relevant accents depending on the device.
- Voice in (server): `/api/transcribe` forwards the audio to OpenRouter’s audio transcriptions endpoint with model `openai/whisper-large-v3`; without a key it returns guidance to use the browser microphone.
- Voice out: `/api/tts` sends up to 2,500 characters to ElevenLabs (model `eleven_multilingual_v2`, stability 0.45, similarity_boost 0.75, voice id from `ELEVENLABS_VOICE_ID`, default `21m00Tcm4TlvDq8ikWAM`). If no key or an error, the client falls back to the device speech synthesizer — the field desk always has a voice.
- Field briefs return a speakable field (markdown stripped) so the browser can read the card aloud in Twi, Ewe, Ga, Hausa or English.

System 11 — Ghana-weighted web search

Where: lib/search.ts. Every query is suffixed “Ghana health”. Primary path: Tavily advanced search restricted to eleven trusted domains — ghs.gov.gh, moh.gov.gh, who.int, afro.who.int, cdc.gov, reliefweb.int, promedmail.org, ourworldindata.org, ghanahealthservice.org, statsghana.gov.gh, mofa.gov.gh — max 6 hits, snippet 320 chars. Without a Tavily key, the fallback parses DuckDuckGo HTML (result__a / result__snippet) and unwraps uddg redirects. Search results feed the Intelligence chat (with inline citations) and the morning sitrep; the briefing prompt forbids the model from using anything outside the provided hits.

System 12 — Climate desk

Where: lib/weather.ts, app/api/weather/route.ts, app/climate. Ten Ghana watch-cities (Accra, Kumasi, Takoradi, Cape Coast, Koforidua, Ho, Tamale, Bolgatanga, Wa, Sunyani) are fetched from OpenWeather in parallel (Promise.all) with metric units, mapping each city back to its region. Without a key or on network failure, the desk shows clearly-labelled placeholder rows (28 °C, 78% humidity, “connect OpenWeather”) — the desk never pretends placeholders are live data. The morning sitrep (/api/briefing) combines the national snapshot, weather rows and search hits into one 220-word brief, again with the “use only provided numbers” constraint.

System 13 — DHIMS2 / IDSR extraction and quality

Where: lib/dhims2.ts, app/api/series. The parser accepts CSV, TSV or semicolon-separated files and infers columns by normalised header names: date columns (date, week_ending, weekending, period, epiweek, epi_week, week, startdate, enddate), case columns (cases, new_cases, confirmed, suspected, count, value, total, notifications), plus optional region/district/disease columns. Dates are parsed to ISO, including YYYY-Www epiweeks (ISO week rule: Jan 4). Then:

- Negative cells are clipped to 0 and counted (typical DHIMS2 correction).
- Duplicate week keys are summed and reported.
- Missing weeks between first and last date are enumerated; completeness = present / (present + missing).
- Warnings fire when completeness < 85%, negatives were clipped, duplicates existed, or fewer than 8 weeks (too short for a 4-week test window).
- Uploading replaces any previous official series for the same disease/region/district, records the quality log, writes the append-only event, and asks OpenRouter for a 120-word DHIMS2 manager’s review (fitness, gaps, three cleaning actions) — never claiming the extract is error-free.
- A template CSV (12 weeks of Greater Accra cholera) is downloadable from ?template=1 so districts know the exact shape expected.

Once an official series is loaded, the forecast engine uses it instead of the demonstration series and the diagnostics block reports source: “official” — the desk is explicit about which data made the number.

System 14 — Reporting-delay nowcast

Where: lib/nowcast.ts. DHIMS2-style reporting is incomplete for the most recent weeks, so the desk treats the last weeks as partial reports rather than true drops. A published-style completeness curve is used: the most recent week is ~55% complete, then 78%, 91%, 97%, 100% (DEFAULT_P = [0.55, 0.78, 0.91, 0.97, 1], applied for lags ≤ 3). nowcast = observed / completeness; the band widens as completeness falls (band = 0.28 × (1 – completeness); high uses ×1.4). The caveat is returned with every result: “Last weeks are treated as incomplete reports, not as a true drop in disease... A nowcast is still an interval.” This is the honest bridge between the incomplete latest weeks and the forecast that starts from them.

System 15 — Field briefs in five languages

POST /api/field-brief returns a 90-word field card for a CHPS compound or district officer: disease, region, district, latest count, z, next-week interval, nowcast, source, and three numbered actions — in English, Twi, Ewe, Ga or Hausa (languageInstruction() appends “reply in X, then a short English recap”). Offline fallback cards exist for every language (e.g. the Twi card: “Afie ns■■m (offline). Hw■■ nnaw■■twe yi line list...”), so a district without any AI key still receives a useful, honest card. The card is also returned as speakable text for the browser voice (System 10).

System 16 — Admin, CMS, auth and the signed audit

- Auth: POST /api/admin/login verifies email + bcrypt password with timingSafeEqual, then issues an HMAC-SHA256-signed session cookie (ohg_admin, 7-day expiry, HttpOnly, SameSite=Lax, Secure in production). All admin routes check the session.
- CMS: /api/admin/cms reads/writes site copy, Ghana-flag colours, nav visibility and knowledge cards; every change is logged to activity and the event archive.
- Records: the admin desk can delete documents, chats, forecasts and knowledge items (audited).
- Signed audit: /api/admin/audit exports a full pack (audits, activity, documents, chats, forecasts, officialSeries) signed with HMAC-SHA256 (lib/sign.ts); a POST verifies a pack against the server secret — the One Health Secretariat can therefore prove a pack was not altered since export.
- Event archive: /api/archive shows stats and the recent events and allows a raw .jsonl download.

System 17 — The eleven desks (page map)

Page	Role on the desk
/ (Home)	National briefing surface. Pillars, priority signals, honest limits, Ghana flag bar.
/forecast	Leakage-safe ensemble, 2–12 week horizon, interval not a point, official CSV overlay, delay nowcast, AI briefing, model scoreboard.
/surveillance	All 13 signals: latest count, z-score, next-week interval. Watch vs investigate.
/climate	Parallel OpenWeather watch-cities plus a sitrep that may mix weather, search, and the ensemble.
/intelligence	Ghana-locked chat, web search, voice in/out, language, live ensemble attached to the prompt.
/vision	Vision Lab: PDF, Word, Excel, CSV, photos. Local text extract + multi-model vision. Redaction first.
/extracts	DHIMS2 / IDSR weekly CSV. Quality log, completeness, OpenRouter review, then the forecast uses that series.
/field	Ninety-word CHPS card in English, Twi, Ewe, Ga or Hausa. Speak it.
/regions	Sixteen official regions plus representative MMDAs. Unit of analysis is region-week.
/workbook	This workbook — the modified Phase 2 teaching contract and the build log of every system.
/admin	Wix-style CMS: copy, colours, nav, knowledge, records, API probe, signed audit, event log.

System 18 — Hosting on Render

render.yaml defines a free web service and a free Postgres. scripts/start.cjs spawns next start -H 0.0.0.0 -p \$PORT and forwards SIGTERM/SIGINT so Render can stop the service cleanly. The build is npm ci --include=dev &&

npm run build; health is /api/health. Frankfurt is the nearest common region to Accra. Honest operational notes: the free web sleeps after ~15 minutes (next visitor pays a cold start); free Postgres expires on the hobby clock; huge vision PDFs can exhaust 512 MB RAM; some sandboxes reset outbound TLS so provider probes fail there even with valid keys — the Admin → API desk on the live host is the authoritative probe.

Required environment (server-side only)

Variable	Purpose	Required
OPENROUTER_API_KEY	Chat, vision, briefing, extract review, field card, optional Whisper	For live AI
ADMIN_EMAIL / ADMIN_PASSWORD / ADMIN_NAME	CMS login (script-hashed; change the default)	Yes
SESSION_SECRET	HMAC key for admin sessions and audit signatures	Yes
NEXT_PUBLIC_SITE_URL	Safe public site URL used for OG metadata and Referer	Yes
TAVILY_API_KEY	Cited web search (else DuckDuckGo fallback)	Optional
ELEVENLABS_API_KEY (+ VOICE_ID)	Spoken briefings	Optional
OPENWEATHER_API_KEY	Climate watch-cities	Optional
DATABASE_URL	Postgres snapshot + event log for durability across web sleep	Recommended
OPENROUTER_MODELS	Extra model slugs tried first (comma-separated)	Optional

System 19 — Security posture

- Secrets exist only in server environment variables. Nothing secret is ever exposed through NEXT_PUBLIC_* or returned by any public route — /api/health returns only masked keys (first 8 + last 4).
- Redaction runs before any model sees text (System 9); the audit records how many redactions fired.
- Admin sessions are HMAC-signed with timing-safe comparison; passwords are script-hashed; audit packs are HMAC-signed so export tampering is detectable.
- Route handlers never read files from the filesystem paths provided by clients; uploads are classified by extension/mime and content, not executed.
- The Ghana-locked system prompt forbids the model from inventing official circulars, case counts or lab results.

System 20 — Ethics and what we refuse

- No “error-free” future. That sentence is scientifically false and is rejected in product copy.
- No individual clinical diagnosis from a photograph.
- No outbreak confirmation from a z-score — an alert is a prompt to investigate.
- No patient name, folder number, phone, or small-cell paediatric count in a prompt.
- No API key in NEXT_PUBLIC_* or client JavaScript.
- No global operations centre. Ghana only, unless the user is comparing an imported border risk.
- OpenRouter may draft a briefing; it may not decide that an intervention launches. The officer owns the decision.

PART IV — The classroom companion notebook

Where: notebooks/ghana_one_health_forecasting.py. Run cell-by-cell in Google Colab or locally. It is the reproducible proof behind the live demonstration: same science, different stack (pandas / scikit-learn / statsmodels instead of the TypeScript engine).

Module-by-module walkthrough

Notebook cell(s)	What it does (verified from source)
Setup	pip-installs pandas, numpy, matplotlib, seaborn, scikit-learn, statsmodels; sets COUNTRY = "Ghana" (the code.txt default was South Africa — fixed).
Module 4 — Ingest	Reads OWID COVID-19 CSV, keeps location/date/new_cases/new_deaths/population, filters to Ghana, sorts by date, reports shape, missing values and duplicate dates.
Module 5 — Clean	Clips negatives to 0, resamples to weekly sums, interpolates at most 2 internal missing weeks with linear interpolation (limit_area="inside"), flags imputed gaps.
Module 6 — EDA	Plots raw weekly cases with a 4-week rolling mean — the same first chart the live desk shows for any disease-region pair.
Module 7 — Diagnostics	Seasonal decomposition (period 52), Augmented Dickey–Fuller test, ACF and PACF plots — the evidence for lags and differencing decisions.
Module 8 — Baselines	Naive (shifted), seasonal naive (lag 52), and 4-week moving average — all computed so they cannot see the target week.
Module 9 — Split	Chronological train/validation/test with TimeSeriesSplit (walk-forward); nothing after the cut informs training.
Module 10–12 — Features and models	Lags and rolling features engineered BEFORE the split (the leaky merge in code.txt is fixed); RandomForestRegressor; SARIMAX fitted on the train split only, forecasting len(test) — the future-week leak is fixed.
Module 13–14 — Evaluation	MAE, RMSE and sMAPE against the naive baseline on the same held-out window; intervals are printed, never bare points.
Module 15 — Early warning	z-score threshold alerts (the syntax-error print is fixed); alerts are investigation prompts, not confirmations.
Module 16–20 — Interpretation, AI, ethics, capstone	Feature-importance caveats, AI-as-tutor-not-analyst rules, the no-identifiers rule, and the capstone checklist that the live desk implements.

How the notebook maps to the production engine

Concept	Notebook (Python)	Production (TypeScript)
Baselines	pandas shifts and means	naive / seasonal-naive / 4-week MA in lib/forecast.ts
ML model	RandomForestRegressor + SARIMAX (statsmodels)	hand-written ridge (closed form) + 24-tree forest + Holt
Split	TimeSeriesSplit, chronological	single 20%-hold-out (min 6, max 26 weeks)
Features	lags + rolling after shift	lag_1–4, rolling mean/std (shifted), week, month, rainy, harmattan
Evaluation	MAE/RMSE/sMAPE vs naive	same metrics on the same held-out window
Alerts	z-threshold print	8-week z + CUSUM (k=0.5, h=5), watch/alert/severe

Concept	Notebook (Python)	Production (TypeScript)
Data	OWID Ghana COVID	13 Ghana signals × 16 regions, demo until official extracts load

KEY TAKEAWAY — Two independent implementations of the same scientific contract (Python notebook and TypeScript engine) is the strongest validation this project has: when both beat naive on chronological windows and both print intervals, the science is doing the work, not the library.

PART V — Appendices

Appendix A — Worked example: malaria, national, 4-week horizon

This is real output captured from lib/forecast.ts on 2026-08-19 (seed “malaria:national:v4”), not a mock-up. It is a demonstration series, clearly labelled, until official DHIMS2 extracts are loaded.

Inputs

- Disease: malaria. Region: national (all regions). Horizon: 4 weeks. Series: 260 generated weekly points (min 2,667, max 15,499).
- Chronological split: train through 2026-02-16; held-out test from 2026-02-23 (26 weeks).
- Latest observed week: 2026-08-17 → 5,108 confirmed cases.

Model scoreboard on the held-out window (MAE / RMSE / sMAPE)

Model	MAE	RMSE	sMAPE	Next-week point (interval)
Ridge	401.2	774.3	8.4%	4,992 (3,894–6,475)
Naive (last week)	508.0	846.4	10.6%	5,108 (3,678–7,039)
Random forest	593.2	871.4	12.4%	5,216 (4,671–6,248)
4-week moving average	666.2	1,061.4	13.6%	5,331 (4,051–7,058)
Seasonal naive (52w)	917.6	1,297.6	20.2%	4,855 (3,399–6,821)
Holt linear trend	2,815.0	3,107.7	84.1%	5,428 (4,017–7,333)

Ensemble

Ranking by MAE selects ridge + naive + forest — the three lowest-MAE members. Ensemble MAE = 500.8. Four-week outlook: 5,105 → 5,232 → 5,222 → 5,160 (week of 2026-08-24 first), interval low 3,678 and interval high 7,039 across the horizon. In the workbook rule’s language: “about 5,100 cases next week (interval 3,678–7,039).”

Alerts in the recent history (z over the 8-week baseline)

Week	Observed	z	CUSUM	Source	Level
2026-07-06	5,331	2.18	0	zscore	watch
2026-07-13	5,413	2.02	0	zscore	watch
2026-08-03	5,889	2.70	0.242	zscore	alert

Latest diagnostics

Eight-week mean 5,236.75, standard deviation 326.22, latest 5,108 → $z = -0.39$, latest CUSUM = 0. The desk’s narrative reads: “Malaria confirmed cases last week: 5,108. Eight-week mean 5,237 ($z = -0.39$).” The ensemble outlook averages about 5,180 with the interval stated. Feature caveats: lag_1 dominates (0.31) — the strongest statistical predictor, not a causal claim.

What a reader should take from this

Naive and ridge are the hardest to beat for a smooth, strongly seasonal series like national malaria — exactly what the scientific contract predicts. Holt’s sMAPE of 84% shows what happens when a trend model runs on a seasonal series it cannot shape. The interval on every line is the point of the exercise.

Appendix B — The full OpenRouter chain, grouped by three

Chat chain as actually chunked (each group is one request; the first group that answers wins):

Group	Slugs (sent together, in this order)
1	openai/gpt-4.1-mini · google/gemini-2.5-flash · openai/gpt-4o-mini
2	openai/gpt-4.1 · google/gemini-2.5-pro · openai/gpt-4o
3	anthropic/claude-sonnet-4 · anthropic/claude-3.5-sonnet · deepseek/deepseek-chat
4	meta-llama/llama-3.3-70b-instruct · mistralai/mistral-large-2411
5 (free)	meta-llama/llama-3.3-70b-instruct:free · google/gemma-3-27b-it:free · qwen/qwen-2.5-72b-instruct:free
6 (free)	mistralai/mistral-7b-instruct:free · nousresearch/hermes-3-llama-3.1-405b:free
7 (auto)	openrouter/auto

Vision chain: Gemini 2.5 Flash → GPT-4o-mini → Gemini 2.5 Pro → GPT-4.1 → GPT-4o → Claude Sonnet 4 → Claude 3.5 Sonnet → Qwen2.5-VL 72B :free → Gemma 3 27B :free (same three-per-group rule). A 402 (empty OpenRouter balance) stops the paid groups and retries the :free set plus openrouter/auto.

Appendix C — Render deployment and secrets

- 1 Push the branch (this session: arena/01a018b0-one-health).
- 2 Create a Web Service from render.yaml (or the Blueprint) — Node 20+, free tier.
- 3 Build: `npm ci --include=dev && npm run build`. Start: `npm start` (scripts/start.cjs binds 0.0.0.0 and honours \$PORT).
- 4 Set secrets in the Environment tab: OPENROUTER_API_KEY, ADMIN_EMAIL, ADMIN_PASSWORD, SESSION_SECRET, NEXT_PUBLIC_SITE_URL, and optionally TAVILY_API_KEY, ELEVENLABS_API_KEY, OPENWEATHER_API_KEY, DATABASE_URL, OPENROUTER_MODELS.
- 5 Open /api/health — expect openrouter: true and masked key; open /api/diagnostics for the live probes.
- 6 Change the default ADMIN_PASSWORD immediately; keep the database backed up off-site for multi-decade retention.

Post-deploy verification (from the Render forensic pass)

- 1 Open /api/health — expect openrouter: true and openrouterMaxModelsPerRequest: 3.
- 2 Open Admin → API desk → probe — OpenRouter should answer OPENROUTER_OK with a live model name, not a 400.
- 3 Intelligence: send “Greater Accra cholera watch, no names.” Expect a model name in the response, not the offline card.
- 4 Climate: Accra temperature should be live (not a placeholder).
- 5 Field brief: draft and speak — browser voice if the ElevenLabs key is restricted.

What is still not “perfect” (stated, not hidden)

- Free web sleeps after ~15 minutes — the next visitor pays a 30–60 s cold start.
- Free Postgres expires on the hobby clock (~30 days) — multi-decade retention needs a kept, backed-up database.
- Free-model AI is rate-limited.
- Huge vision PDFs can exhaust 512 MB RAM.
- ElevenLabs restricted keys cannot read account metadata; speech itself still works.
- Demonstration series are epidemiologically shaped, not official — replace them before operational use.

Appendix D — Glossary (unchanged science)

Surveillance — ongoing collection and analysis of health data. Outbreak / epidemic / pandemic — local, regional, global rise above expectation. Incidence — new cases in a period. Time series, trend, seasonality, stationarity, autocorrelation, lag, baseline, chronological train/validation/test, leakage, overfitting, MAE/RMSE/sMAPE, prediction interval, feature importance (not causation), early warning (investigation prompt). Production terms: region-week (unit of analysis), demonstration series (shaped, not official), official series (loaded DHIMS2/IDSR extract), nowcast (estimate of an incomplete recent week), CUSUM (accumulating deviation detector), redaction gate (identifier stripping before any model sees text), walk-forward (rolling evaluation on chronological windows).

Appendix E — Sources

OWID COVID-19 · WHO GHO and Disease Outbreak News · Ghana Health Service IDSR / DHIMS2 (official extracts, not scraped) · pandas, scikit-learn, statsmodels · OpenRouter routing documentation · original Phase 2 workbook (2026-08-19 Word export) · the ONE HEALTH GHANA repository itself (this workbook quotes its own code).

Verify every live URL before a training delivery. Demonstration series on the website are epidemiologically shaped for Ghana and must be replaced before operational decisions.

Appendix F — Verification checklist (self-test)

scripts/selftest.cjs exercises the running desk; every check must pass before a training delivery:

- health reports ok and capability flags; public CMS config loads.
- forecast snapshot returns at least five signals; a malaria/national POST returns an ensemble with points and a numeric latestCusum.
- admin rejects bad credentials with 401 and accepts the configured login.
- official series upload parses a CSV, and the DHIMS2 quality log reports completeness.
- redaction fires: a chat containing “Patient: Ama Mensah phone 0244123456” must NOT return the phone number.
- nowcast interval is ordered ($\text{high} \geq \text{nowcast} \geq \text{low}$).
- the event archive grows after a mutation.
- a Twi field brief returns non-empty text.

- all public pages return 200.

This page ends the workbook. The science contract from Module 1 is the last word: understand → implement → validate → let AI assist → criticise → improve → interpret — and always print the interval.